

HALCYON

Technical Documentation

HALCYON is a Flask-based web application for exploring relationships between weather patterns and health outcomes across U.S. counties (2013–2023), using data from NOAA, CDC Places, and the U.S. Census Bureau.

Part 1 — General

Pages

Route	Method	Description
/	GET	Landing page and project overview.
/explore	GET	Filter and visualize data by year, county, and climate type.
/predict	GET	Select a county and generate health outcome predictions.
/docs	GET	This documentation page.

Data Sources

NOAA — National Oceanic and Atmospheric Administration

This data comes from the National Ocean and Atmospheric Administration (NOAA) Global Summary of the Year, and supplies the HALCYON app with temperature and weather data from across the United States. This data set provides 16 variables for this app, and the geographic basis for a 17th.

<https://www.ncei.noaa.gov/cdo-web/>

CDC Places

The CDC Places dataset supplies the HALCYON app with health data at the county level, including the app's predicted outcome variables: stroke incidence, mental health quality, physical health, asthma incidence, and sleep.

<https://www.cdc.gov/places/tools/data-portal.html>

U.S. Census Bureau

The U.S. Census Bureau dataset supplies the HALCYON app with demographic information at the county level, covering twelve variables used to eliminate non-climate variations between health outcomes.

<https://www.census.gov/data/developers/data-sets/acs-5year.html>

Dataset — 6,646 rows · 39 columns

The app loads `merged_final_transformed.csv` at startup. The dataset merges three sources: NOAA climate station data, CDC Places health estimates, and U.S. Census socioeconomic indicators, which are linked at the county level by year.

IDENTIFIERS

Column	Description
<code>year</code>	Year of observation (2013–2023)
<code>County_name</code>	U.S. county name — not unique across states, use with <code>StateAbbr</code>
<code>StateAbbr</code>	Two-letter state abbreviation
<code>CountyFIPS</code>	5-digit FIPS code uniquely identifying each county
<code>STATION</code>	NOAA station identifier
<code>STATION_NAME</code>	NOAA station name
<code>LATITUDE</code>	Station latitude (decimal degrees)
<code>LONGITUDE</code>	Station longitude (decimal degrees)
<code>ELEVATION</code>	Station elevation (meters)
<code>climate_type_short</code>	Abbreviated Köppen climate classification — derived from station location, used as a categorical feature

NOAA — CLIMATE VARIABLES

Column	Description
<code>TAVG</code>	Average annual temperature (°F)
<code>TMAX</code>	Average annual maximum temperature (°F)
<code>TMIN</code>	Average annual minimum temperature (°F)
<code>EMXT</code>	Extreme maximum temperature — highest recorded in the year (°F)
<code>EMNT</code>	Extreme minimum temperature — lowest recorded in the year (°F)
<code>CLDD</code>	Cooling degree days — cumulative heat above 65°F baseline
<code>HTDD</code>	Heating degree days — cumulative cold below 65°F baseline
<code>DX90</code>	Number of days with maximum temperature $\geq 90^{\circ}\text{F}$
<code>DX70</code>	Number of days with maximum temperature $\geq 70^{\circ}\text{F}$
<code>DX32</code>	Number of days with maximum temperature $\leq 32^{\circ}\text{F}$ (freeze days)
<code>DT32</code>	Number of days with minimum temperature $\leq 32^{\circ}\text{F}$ (frost days)

Column	Description
PRCP	Total annual precipitation (inches)

CDC PLACES — HEALTH OUTCOMES (CRUDE PREVALENCE %, AGE-ADJUSTED)

Column	Description
CASTHMA	Current asthma prevalence
STROKE	Stroke prevalence
MHLTH	Frequent mental distress prevalence (14+ poor mental health days/month)
PHLTH	Frequent physical distress prevalence (14+ poor physical health days/month)
SLEEP	Insufficient sleep prevalence (<7 hours/night) — sleep data collected every two years

U.S. CENSUS — SOCIOECONOMIC INDICATORS

Column	Description
total_population	Total county population
pct_female	Percentage of population identifying as female
pct_male	Percentage of population identifying as male
median_age	Median age of county population
median_household_income	Median household income (USD)
pct_bachelors_plus	Percentage with a bachelor's degree or higher
pct_graduate_degree	Percentage with a graduate or professional degree
pct_less_than_hs	Percentage without a high school diploma
pct_white	Percentage identifying as white (non-Hispanic)
pct_black	Percentage identifying as Black or African American
pct_asian	Percentage identifying as Asian
pct_hispanic	Percentage identifying as Hispanic or Latino

Known Issues & Limitations

While every effort has been made to make the models as robust as possible, several limitations exist that should be taken into account before drawing any conclusions from the predicted values.

Data

The dataset used to train the models only contained data for 10 years, which is a relatively short time period for slow-moving variables such as health outcomes and annual weather patterns. As a result, models are heavily biased

towards events in this short time period and may not accurately reflect more long-term trends.

Models

Not all models are equally robust. While models for Stroke, Mental Health, and Asthma show lower error rates for the available data, Physical Health and Sleep both perform poorly and findings from those two models should be assessed with heavy qualifications.

Confidence Intervals

Due to limited data availability, a trade-off was made during model training between better point predictions and better accuracy of confidence intervals by retraining models on a set of the data that was used to calibrate prediction intervals. Diagnostics reveal that this has not significantly affected CI coverage, with coverage only being poor for the two weaker models (Physical Health and Sleep), but users should still assume that existing confidence intervals may provide more optimistic coverage than may be desired.

Code Sources

Code for the custom RFF function was adapted from Drew Dimmery's lecture material for the 2026 Data Structures and Algorithms course, available at the class GitHub repository:

<https://github.com/hertie-ml-26/lecture-slides/tree/main/src/ml26>

In addition, AI coding assistance was used for several sections of the app code, in particular HTML formatting for some of the interactive features. All such instances have undergone detailed reviews from the developers to ensure that the AI-generated code reflects the design and user functionality, design and user interface goals of the app.

Part 2 — Technical

Architecture Overview

The ML pipeline is split across eight files. No external ML libraries (e.g. scikit-learn) are used for the core model; all components are implemented from scratch using NumPy and Pandas. XGBoost is used for the **SLEEP** target only.

File	Class / Function	Responsibility
<code>splitter.py</code>	Splitter, <code>prepare_data()</code>	Drops non-features, creates time-series CV folds and the final train/test split
<code>preprocessing.py</code>	Preprocessor	Coordinates encoding, imputation, and scaling via composition — owns instances of <code>OneHotEncoder</code> , <code>SimpleImputer</code> , <code>StandardScaler</code>
<code>random_fourier_features.py</code>	<code>RFFRidgeRegression</code>	Primary model (all targets except SLEEP) — approximates KRR using Random Fourier Features; supports conformal prediction intervals
<code>xgboost_wrapper.py</code>	<code>XGBoostWrapper</code>	Model for SLEEP only — wraps <code>XGBRegressor</code> to match the RFF interface; uses quantile regression for prediction intervals, optionally corrected with conformal calibration

File	Class / Function	Responsibility
<code>cross_validator.py</code>	CrossValidator	Runs CV across time-series folds; creates a fresh Preprocessor per fold to prevent data leakage
<code>assessment.py</code>	Assessment	R ² score and MSE — used for hyperparameter tuning (MSE) and reporting (R ²)
<code>train.py</code>	<code>tune_and_evaluate()</code> , <code>save_model()</code> , <code>load_model()</code>	Entry point — runs grid search, fits final model, calibrates, saves .pkl bundles loaded by the Flask app
<code>scenarios.py</code>	<code>generate_scenario()</code>	Builds synthetic future feature rows for a county under a climate scenario; used by the Flask app to generate 10-year predictions

Training Pipeline

A separate model is trained for each health target by `tune_and_evaluate()` in `train.py`. The steps are the same for every target:

Step	What happens	Module
1. Data prep	<code>prepare_data()</code> drops all other health outcomes and non-feature identifiers, then splits into X, y, and year labels	<code>splitter.py</code>
2. Fold creation	<code>Splitter.time_series_splits()</code> builds expanding-window CV folds (2014->2015, 2014-2015->2016, ... 2014-2021->2022); 2023 is held out as the test set throughout	<code>splitter.py</code>
3. Grid search	CrossValidator scores each hyperparameter combination using the last 3 stable folds; early folds are skipped because the training set is too small before 2018	<code>cross_validator.py</code>
4. Final fit	Best model is fit on 2013-2021, calibrated on 2022, retrained on 2013-2022 data, and then evaluated on the held-out 2023 test set	<code>train.py</code>
5. Save	Model + fitted preprocessor are pickled together as <code>models/{TARGET}.pkl</code> for the Flask app to load	<code>train.py</code>

Preprocessing

`Preprocessor` coordinates three components via composition (not inheritance), applied in order: encode -> impute -> scale. A fresh instance is created inside each CV fold to prevent statistics from leaking between folds.

Column	Description
<code>OneHotEncoder</code>	Converts <code>climate_type_short</code> (the only categorical column) into binary indicator columns. Drops the first category to avoid multicollinearity. Fit on training data only — unseen categories in val/test are encoded as all-zeros.
<code>SimpleImputer</code>	Replaces NaNs with column means learned from training data. Most columns have <10% missing.

Column	Description
StandardScaler	Standardises each feature to mean=0, std=1 using $z = (x - \mu) / \sigma$. Required because the RFF kernel is sensitive to feature scale. Constant columns (std=0) are left unchanged.

The `fit_transform()` method is used on training data; `transform()` applies the saved statistics to validation and test data. This separation is the primary guard against data leakage.

Random Fourier Features (RFF) Model

Kernel Ridge Regression (KRR) solves $\alpha = (K + \lambda I)^{-1} \cdot y$ exactly, but requires building and inverting an $n \times n$ kernel matrix — $O(n^3)$ in time and $O(n^2)$ in memory. For large datasets this becomes infeasible. Random Fourier Features approximate the kernel implicitly, reducing the problem to ordinary ridge regression on a low-dimensional feature map Z .

Feature map

R random frequencies W are drawn from $N(0,1)$ and phases B from $U(0, 2\pi)$, then:

$$Z = \sqrt{2/R} \cdot \cos(\sigma \cdot X \cdot W^T + B)$$

This gives an (n, R) matrix Z such that $Z \cdot Z^T$ approximates K for the Gaussian kernel with bandwidth σ .

Fitting — coefficients are solved directly using ridge regression on Z :

$$\text{coef} = (Z^T \cdot Z + \lambda I)^{-1} \cdot Z^T \cdot y$$

Prediction

$$y_{\text{pred}} = Z_{\text{test}} \cdot \text{coef}$$

Conformal prediction intervals

After fitting, `calibrate()` stores nonconformity scores (absolute residuals) on the 2022 held-out calibration set. At inference, `predict_interval()` computes the finite-sample corrected quantile from Angelopoulos & Bates (2022):

$$q_{\text{level}} = \text{ceil}((1 - \alpha) \cdot (n_{\text{cal}} + 1)) / n_{\text{cal}}$$

The prediction interval is $[y_{\text{pred}} - q, y_{\text{pred}} + q]$. This gives a coverage guarantee: at least $(1 - \alpha)$ of test points will fall within the interval.

Hyperparameters

Parameter	Description	Grid searched values
<code>sigma</code>	Kernel bandwidth — controls how quickly similarity decays with distance	0.1, 0.5, 1.0, 2.0, 5.0
<code>lamb (λ)</code>	Regularisation strength — higher values reduce overfitting	1e-5, 1e-4, 1e-3, 1e-2, 1e-1
<code>R</code>	Number of random Fourier features — higher R gives a better kernel approximation at the cost of compute	200, 500, 1000

XGBoost Model (SLEEP only)

The **SLEEP** target uses **XGBoostWrapper** instead of RFF. SLEEP data is collected every two years, making the training set smaller and noisier — gradient boosting is more robust to this than the kernel approximation.

XGBoostWrapper trains three models in parallel: a median model (**reg:squarederror**) for point predictions, and lower/upper quantile models (**reg:quantileerror** at $\alpha=0.025$ and 0.975) producing a base 95% quantile interval. These bounds are then optionally narrowed or widened by conformal calibration — the correction takes $\min(\text{lower}, y_{\text{pred}} - q)$ and $\max(\text{upper}, y_{\text{pred}} + q)$, where q is the same finite-sample corrected residual quantile used in RFF — to achieve the requested coverage level (default 90%).

Parameter	Description	Grid searched values
<code>n_estimators</code>	Number of boosting rounds	100, 300
<code>max_depth</code>	Maximum tree depth — controls model complexity	4, 6, 8
<code>learning_rate</code>	Shrinkage factor applied to each tree's contribution	0.01, 0.05, 0.1

Assessment

The **Assessment** class in `assessment.py` implements two metrics:

Metric	Formula	Used for
MSE	<code>mean((y_test - y_pred)^2)</code>	Hyperparameter selection during grid search, minimized to pick the best configuration
R ²	<code>1 - (RSS / TSS)</code>	Final reporting — 1.0 is perfect, 0 = predicting the mean, <0 = worse than the mean

Target Outcomes

A separate model is trained for each of the five health outcomes. All are crude prevalence percentages from CDC Places.

Outcome	Column	Model
Asthma	<code>CASTHMA</code>	RFF
Mental distress	<code>MHLTH</code>	RFF
Physical distress	<code>PHLTH</code>	RFF
Stroke	<code>STROKE</code>	RFF
Insufficient sleep	<code>SLEEP</code>	XGBoost

Model Diagnostics

All diagnostic charts are generated by `diagnostics.ipynb`. The notebook loads saved `.pkl` models via `load_model()` from `train.py` and reproduces the exact 2023 test split used during training.

R² and MSE

Model performance varies across outcomes. Stroke has the strongest predictive performance, with an R² close to 1 and low MSE. Asthma and mental health show modest R² values (~0.5). Physical health performs poorly (negative R²), indicating the model underperforms a simple mean prediction. Sleep, modelled with XGBoost, shows moderate predictive performance.

Model comparison — custom RFF vs sklearn KRR vs sklearn RFF

Performance is broadly consistent across all three approaches. The custom RFF and sklearn RFF approximation produce similar R² and MSE values and are substantially faster than exact sklearn KRR. The main divergence is for Physical Health, where the custom RFF shows a slightly more negative R², though all three models perform weakly on that target. Runtime figures exclude hyperparameter tuning and reflect final-pipeline inference only.

Residuals vs predicted

Residuals are largely randomly scattered around zero for Asthma, Mental Health, and Stroke, suggesting no systematic bias for those models. Physical health shows a downward bias, confirming model weakness: the model is underpredicting for lower predicted cases, which constitute the majority. Residuals for most targets skew positive, indicating predictions tend to fall below actual values. Stroke residuals are closest to zero, consistent with its high R².

Predicted vs actual

Points close to the diagonal red line indicate accurate predictions. Stroke, Asthma, and Mental Health cluster tightly along the diagonal. Physical Health shows systematic underprediction with points lying above the line throughout the range. Sleep displays range compression: the slope of the predicted-vs-actual curve is much shallower than the ideal diagonal, meaning the model struggles to capture the full variation in sleep outcomes.

90% prediction intervals

Shaded bands are 90% conformal prediction intervals; red dots are actuals sorted by true value. Stroke, Asthma, and Mental Health show strong coverage, with the bands capturing actuals even at the extremes. Physical Health intervals cover all actuals but predictions remain systematically low. Sleep intervals fail to capture actuals at the tails, reflecting the range-compression issue seen in the predicted-vs-actual chart.

Testing

The test suite covers all core pipeline modules across three layers: unit tests, integration tests, and one end-to-end test. Tests are organised into a three-tier folder structure and run with pytest. Each test module imports directly from [app/functions](#) so no installation step is required.

Unit tests

Assessment — [test_r2_score.py](#), [test_mean_squared_error.py](#)

Verifies the two metric implementations against known hand-computed values. Key cases covered:

- **R²** — returns 1.0 for perfect predictions; returns 0.0 when predicting the mean; can be negative for worse-than-mean predictions; never exceeds 1.0; handles the constant-target edge case (TSS=0) — returns 1.0 for a perfect prediction and 0.0 for an imperfect one, avoiding division by zero
- **MSE** — returns the correct average squared error against a hand-computed value; returns 0.0 for perfect predictions; handles negative input values correctly; verifies the quadratic penalty — an error of 3 is penalized 9x

more than an error of 1

Preprocessing — [test_simple_imputer.py](#), [test_standard_scaler.py](#), [test_one_hot_encoder.py](#), [test_detect_categorical_columns.py](#), [test_preprocessor.py](#)

Each preprocessing component is tested independently and then together through Preprocessor. Key cases:

- [SimpleImputer](#) — replaces NaNs with column means; handles the all-NaN column edge case by falling back to 0 (a NumPy warning is expected and noted in the test)
- [StandardScaler](#) — output has mean=0 and std=1; constant columns (std=0) are left unchanged to avoid division by zero; transform() uses training statistics, not test statistics
- [OneHotEncoder](#) — drops the first category to avoid multicollinearity; numeric columns are preserved unchanged alongside encoded columns; handles multiple categories correctly; unseen categories at inference time become all-zeros rather than raising an error
- [detect_categorical_columns\(\)](#) — returns correct indices for object and category dtype columns, including when both dtypes are present in the same dataframe; returns an empty list when no categorical columns are present
- [Preprocessor](#) — end-to-end: imputes and scales numeric data to mean=0, std=1 with all values finite; mixed numeric/categorical input produces a fully numeric output array; transform() applies the pipeline fitted on training data without refitting

Splitter — [test_splitter.py](#), [test_prepare_data.py](#)

Covers data preparation and fold generation:

- [prepare_data\(\)](#) — keeps the target column in y and drops it from X; drops all other health outcomes and identifier columns ([StateAbbr](#), [County name](#), [CountyFIPS](#), [STATION](#), [STATION_NAME](#)); [year](#) is explicitly **retained** as a feature in X alongside climate and socioeconomic variables; drops rows where the target is NaN and adjusts the returned years array accordingly (critical for SLEEP)
- [Splitter](#) — assigns the last year as the held-out test year and all earlier years as training; [time_series_splits\(\)](#) produces expanding windows with the correct train/val arrays at each fold, verified against exact expected values; [get_test_split\(\)](#) returns the full training set and the held-out test year, verified against exact expected values

CrossValidator — [test_cross_validator.py](#)

Uses a minimal [MeanModel](#) stub to test the CV loop in isolation without depending on the RFF model. [start_fold=0](#) is passed explicitly so all folds are scored — unlike production training, which skips early folds where the training set is too small. Verifies:

- [cross_val_score\(\)](#) returns both [mse](#) and [r2](#) keys
- One score is produced per scored fold
- All scores are finite

KRR baseline — [test_krr.py](#)

Tests the exact Kernel Ridge Regression implementation used as a baseline comparison. Key cases:

- `gaussian_kernel()` — output shape is (n_X, n_Y) for inputs of different sizes; identical points have kernel similarity exactly 1.0 (verified on the diagonal); all kernel values are bounded between 0 and 1
- `fit()` — stores both `coef_` with shape (n_train,) and `X_train_`; retaining training data is required because KRR predictions are computed as a weighted sum over training kernel evaluations
- `predict()` — returns one prediction per test row with the correct output shape
- `prediction_intervals()` — returns three arrays (y_pred, lower, upper) each with the correct shape; bounds are ordered so that $\text{lower} \leq \text{y_pred} \leq \text{upper}$ for all test points

This model is not used in the production Flask app but is evaluated in `diagnostics.ipynb` to benchmark the RFF approximation.

RFF model — `test_random_fourier_features.py`

Six tests covering the full RFF interface:

- `rff_features()` — output shape is (n_samples, R); results are exactly reproducible given the same seed
- `fit()` — coefficients exist after fitting and have shape (R,)
- `predict()` — returns one prediction per test row with the correct output shape
- `predict_interval()` — raises `RuntimeError` matching "calibrate" if called before `calibrate()`; after calibration, returns three arrays (y_pred, lower, upper) each with shape (n_test,); bounds are ordered so that $\text{lower} \leq \text{y_pred} \leq \text{upper}$ for all test points

Scenarios — `test__compute_trend.py`, `test_generate_scenario.py`

Tests for the future projection logic used by the Flask app:

- `_compute_trend()` — returns 0.0 when fewer than 3 data points are available; enforces a positive slope for warming variables — a downward observed trend in TAVG is flipped to +1.0; enforces a negative slope for cooling variables — an upward observed trend in HTDD is flipped to -5.0
- `generate_scenario()` — returns the correct future year range and number of rows; drops all identifier and health outcome columns while retaining `year`, climate variables, and `climate_type_short`; projects warming variables (TAVG: 13.0, 14.0) and cooling variables (HTDD: 35.0, 30.0) forward by exact expected amounts under the middle-of-the-road scenario; raises `ValueError` matching "No data found" when the baseline county/year combination is missing

Integration tests

Basic routes — `test_basic_routes.py`

Uses Flask's test client to verify that all page routes return HTTP 200 and render without errors. Does not require model .pkl files. Routes tested:

- `GET /` — landing page
 - `GET /explore` — explore page
 - `GET /predict` — predict page
 - `GET /docs` — documentation page
-

API routes — [test_map_data_api.py](#), [test_snapshot_api.py](#), [test_summary_api.py](#), [test_timeseries_api.py](#)

Tests the JSON endpoints consumed by the explore page's charts and map:

- [GET /api/map-data?var=](#) — returns HTTP 200 for a valid health variable; response contains exactly the expected top-level keys (data, health_nat_avg, weather_nat_avg, health_var, weather_var, demo_vars, year_start, year_end); each record in data contains fips, county, state, health_val, and population; returns HTTP 400 with {"error": "Invalid variable"} for an unknown variable
- [GET /api/snapshot](#) — takes no parameters; returns HTTP 200 with exactly four dataset-level count keys (states, counties, years, n_vars); all four values are verified to be positive
- [POST /api/summary](#) — accepts a JSON body with a column key; returns HTTP 200 with column and stats (including mean) for a valid column; returns HTTP 400 with {"error": "Invalid column name"} for an unknown column
- [GET /api/timeseries](#) — returns HTTP 200; response contains state and data.health / data.weather keys; the health series contains series, national, var, and stats sub-keys; var matches the requested health variable
- [GET /api/timeseries/heatmap](#) — returns HTTP 200 with variable and data keys; data contains an entry per requested state (verified for CA and NY); returns HTTP 400 for an unknown variable

Prediction page — [test_prediction_page.py](#)

Tests the [/predict](#) route across five tests. Requires saved `.pkl` model files to be present. Verifies:

- Page returns HTTP 200 and contains a form with the exact action `action="/predict" method="POST"`
- Form contains all three required fields: `county_state`, `target`, `scenario`
- All five target outcome values are present in the rendered HTML: `CASTHMA`, `MHLTH`, `PHLTH`, `STROKE`, `SLEEP`
- All four scenario values are present: `low_warming`, `middle_road`, `high_warming`, `very_high_warming`
- A POST request for Alameda County (target `CASTHMA`, scenario `middle_road`) returns HTTP 200, the heading "CASTHMA — ALAMEDA COUNTY, CA", columns for Predicted value (%), Lower 90% CI, and Upper 90% CI, and no prediction error message

End-to-end tests

Navigation flow — [test_navigation_flow.py](#)

Uses **Playwright** to simulate a real browser session against the live app at <http://127.0.0.1:5000>. The Flask server must be started manually before running this test. A single test function covers the full user journey:

- **Home page** — verifies the "Halcyon" heading and "What would you like to do?" are visible
- **Explore page** — navigates via "Explore data →"; verifies the "Exploratory data analysis" heading and that all four filter controls (`#var-health`, `#var-weather`, `#yr-start`, `#yr-end`) and the map SVG (`#map-svg`) are visible
- **Predict page** — navigates via "Open predictor →"; selects state CA and verifies the county dropdown updates with CA-specific options; submits a prediction for Alameda County (target `CASTHMA`, scenario `middle_road`); verifies the heading reads "CASTHMA — ALAMEDA COUNTY, CA", all three result columns are visible, and no prediction error message is shown
- **Docs page** — navigates via "About the data →"; verifies `h1.page-title` contains exactly "Documentation"

Manual exploratory testing was also performed alongside the automated suite to cover interactions not easily captured in Playwright assertions.